You said:

Lab_4_VibeCoding...
PDF

https://github.com/SETAPESU26/12_maze_runner
This is SE Lab 4 go through the two things given and help me complete it step by step u tell me step by step what is suppose to be done how it is suppose to where it is suppose
U tell me one step then i run it if any error i give or output and we go ahead

ChatGPT said:

You said:

PS C:\Users\Owner> git --version
git version 2.53.0.windows.1

ChatGPT said:

You said:

for downlaoding the game what should i do

ChatGPT said:

You said:

Cloning into '12_maze_runner'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 11 (delta 0), reused 0 (delta 0), pack-reused 5 (from 1)
Receiving objects: 100% (11/11), done.

ChatGPT said:

You said:

a new VS window opened with the project files

ChatGPT said:

You said:

Maze Runner

Navigate through a procedurally generated maze to reach the exit using **Pygame**.

Setup

```bash
pip install -r requirements.txt
python main.py
```

Controls

Key	Action
W / UP	Move up
S / DOWN	Move down
A / LEFT	Move left
D / RIGHT	Move right
R	Generate new maze

Tasks to Complete

Task 1: Shortest Path Hint
> Press H to show the solution path highlighted in the maze.

Quick Start Prompt:
```
Add a BFS solver to my pygame maze. When H is pressed, highlight
```

the shortest path from start to exit by drawing colored squares on each cell in the path. Toggle it off on second press.
```

### Task 2: Fog of War
> Only show maze cells within a radius of 3 cells around the player; the rest is dark.

**Quick Start Prompt:**
```
Add fog of war to my maze game. Draw a dark overlay over the whole maze surface, then cut out a circular transparent region around the player using pygame.draw.circle on a surface with SRCALPHA. Reveal only nearby cells.
```

### Task 3: Timer Leaderboard
> Track the top 5 best completion times and display them after solving.

**Quick Start Prompt:**
```
After the player solves the maze, save their time to a list (up to 5 entries, sorted ascending). Display the leaderboard on the win screen. Persist it in a JSON file called leaderboard.json.
```

### Task 4: Larger Maze Difficulty Tiers
> Add Easy (10x8), Medium (15x13), Hard (20x18) options selectable at start.

**Quick Start Prompt:**
```
Add a difficulty select screen before the maze generates. Show three buttons (Easy, Medium, Hard) with different COLS/ROWS values. Let the player click to select, then launch the maze with those dimensions.
```

## Folder Structure
```

```
maze-runner/
├── main.py
├── requirements.txt
├── game/
│   ├── __init__.py
│   ├── game_engine.py
│   ├── maze.py
│   └── player.py
└── README.md
```

Submission Checklist

- [ ] All 4 tasks completed
- [ ] Maze is generated fresh each session (R key)
- [ ] Fog of war renders correctly
- [ ] Timer leaderboard persists to JSON
- [ ] Code reviewed with LLM (include chat link)

ChatGPT said:

You said:

```
PS C:\Users\Owner\12_maze_runner>
/c:/Users/Owner/12_maze_runner/.venv/Scripts/python.exe -m python
-m pip install -r requirements.txt
/c:/Users/Owner/12_maze_runner/.venv/Scripts/python.exe : The term
'/c:/Users/Owner/12_maze_runner/.venv/Scripts/python.exe' is not
recognized as the name of a cmdlet, function, script file, or operable
program. Check the spelling of the name, or if a path was
included, verify that the path is correct and try again.
At line:1 char:1
+ /c:/Users/Owner/12_maze_runner/.venv/Scripts/python.exe -m
python -m  ...
+
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound:
```

```
(/c:/Users/Owner...ipts/python.exe:String) [],
CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

ChatGPT said:

You said:

```
PS C:\Users\Owner\12_maze_runner> python -m pip install -r
requirements.txt
DEPRECATION: Loading egg at
c:\users\owner\appdata\local\programs\python\python311\lib\site-
packages\ply-3.11-py3.11.egg is deprecated. pip 24.3 will enforce this
behaviour change. A possible replacement is to use pip for package
installation.. Discussion can be found at
https://github.com/pypa/pip/issues/12330
Collecting pygame==2.5.2 (from -r requirements.txt (line 1))
  Downloading pygame-2.5.2-cp311-cp311-win_amd64.whl.metadata
(13 kB)
Downloading pygame-2.5.2-cp311-cp311-win_amd64.whl (10.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.8/10.8 MB 5.1 MB/s eta 0:00:00
Installing collected packages: pygame
  Attempting uninstall: pygame
    Found existing installation: pygame 2.6.1
    Uninstalling pygame-2.6.1:
      Successfully uninstalled pygame-2.6.1
Successfully installed pygame-2.5.2

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

ChatGPT said:

You said:

done next

ChatGPT said:

You said:

```
origin  https://github.com/SETAPESU26/12_maze_runner.git (fetch)
origin  https://github.com/SETAPESU26/12_maze_runner.git (push)
```

ChatGPT said:

You said:

for submission we have to use this repo
https://github.com/Khushi-patel05/Team4_SE_Mini_Project which will have a folder called Lab4 and under that all the submission will go

ChatGPT said:

You said:

The Lab4 folder does not exist yet in Team4_SE_Mini_Project.

ChatGPT said:

You said:

PS C:\Users\Owner\OneDrive\Desktop\Lab1-Team4-PES1UG24AM134
this is what it is showing

ChatGPT said:

You said:

```
Cloning into 'Team4_SE_Mini_Project'...
remote: Enumerating objects: 137, done.
remote: Counting objects: 100% (137/137), done.
remote: Compressing objects: 100% (128/128), done.
remote: Total 137 (delta 42), reused 20 (delta 3), pack-reused 0 (from
0)
Receiving objects: 100% (137/137), 6.95 MiB | 10.29 MiB/s, done.
Resolving deltas: 100% (42/42), done.
```

ChatGPT said:

You said:

Directory: C:\Users\Owner\Team4_SE_Mini_Project

Mode	LastWriteTime	Length	Name
d-----	10/19/2026 6:11 PM		Lab1
d-----	10/19/2026 6:11 PM		Lab2
d-----	10/19/2026 6:11 PM		Lab3
d-----	10/19/2026 6:11 PM		Lab4
-a----	10/19/2026 6:11 PM	1295	README.md

ChatGPT said:

You said:

Directory: C:\Users\Owner\Team4_SE_Mini_Project\Lab4

Mode	LastWriteTime	Length	Name
d-----	10/19/2026 6:11 PM		PS4_PES1UG24AM136_Kirthana S

ChatGPT said:

You said:

Directory: C:\Users\Owner\Team4_SE_Mini_Project\Lab4

Mode	LastWriteTime	Length	Name
d-----	10/19/2026 6:13 PM		PS4_PES1UG24AM134_Khushi

ChatGPT said:

You said:

Directory: C:\Users\Owner\12_maze_runner

Mode	LastWriteTime	Length	Name
d--h--	10/19/2026 5:36 PM		.git
d-----	10/19/2026 6:05 PM		.venv
d-----	10/19/2026 5:42 PM		game
-a----	10/19/2026 5:27 PM	34	.gitignore
-a----	10/19/2026 5:27 PM	116	main.py
-a----	10/19/2026 5:27 PM	2264	README.md
-a----	10/19/2026 5:27 PM	15	requirements.txt

ChatGPT said:

You said:

Directory: C:\Users\Owner\12_maze_runner\game

Mode	LastWriteTime	Length	Name
d-----	10/19/2026 6:06 PM		__pycache__
-a----	10/19/2026 5:55 PM	3504	game_engine.py
-a----	10/19/2026 5:56 PM	1182	maze.py
-a----	10/19/2026 5:27 PM	1458	player.py
-a----	10/19/2026 5:27 PM	0	__init__.py

ChatGPT said:

You said:

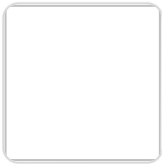
Directory: C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_Khushi

Mode	LastWriteTime	Length	Name

```
----            -------------       ------ ----
d-----       10/19/2026   6:15 PM             game
-a----       10/19/2026   5:27 PM             34 .gitignore
-a----       10/19/2026   5:27 PM            116 main.py
-a----       10/19/2026   5:27 PM           2264 README.md
-a----       10/19/2026   5:27 PM             15 requirements.txt
```

ChatGPT said:

You said:



ChatGPT said:

You said:

On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        game/
        main.py
        requirements.txt

nothing added to commit but untracked files present (use "git add" to track)

ChatGPT said:

You said:

d158699 (HEAD -> main) Initial commit: Add original Maze Runner game

ChatGPT said:

You said:

?? ./

ChatGPT said:

You said:

```
PS
C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_
Khushi> git rev-parse --show-toplevel
C:/Users/Owner/Team4_SE_Mini_Project
PS
C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_
Khushi> git status --short --untracked-files=all
?? .gitignore
?? README.md
?? game/__init__.py
?? game/game_engine.py
?? game/maze.py
?? game/player.py
?? main.py
?? requirements.txt
```

ChatGPT said:

You said:

```
warning: could not open directory
'Lab4/PS4_PES1UG24AM134_Khushi/Lab4/PS4_PES1UG24AM134_Khu
shi/': No such file or directory
fatal: pathspec 'Lab4/PS4_PES1UG24AM134_Khushi/' did not match
any files
```

ChatGPT said:

You said:

A Lab4/PS4_PES1UG24AM134_Khushi/.gitignore
A Lab4/PS4_PES1UG24AM134_Khushi/README.md
A Lab4/PS4_PES1UG24AM134_Khushi/game/__init__.py
A Lab4/PS4_PES1UG24AM134_Khushi/game/game_engine.py
A Lab4/PS4_PES1UG24AM134_Khushi/game/maze.py
A Lab4/PS4_PES1UG24AM134_Khushi/game/player.py
A Lab4/PS4_PES1UG24AM134_Khushi/main.py
A Lab4/PS4_PES1UG24AM134_Khushi/requirements.txt

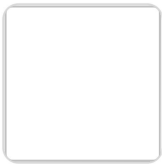
ChatGPT said:

You said:

8c4830c (HEAD -> main) SE Lab 4: Add original Maze Runner game

ChatGPT said:

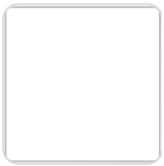
You said:



this is what i see

ChatGPT said:

You said:



this one??

ChatGPT said:

You said:

```
import pygame
import time
from game.maze import generate_maze, CELL
from game.player import Player


FPS = 60
BG = (240, 235, 220)
WALL_COLOR = (40, 40, 60)
EXIT_COLOR = (80, 200, 80)
COLS, ROWS = 15, 13

WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 60

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Runner")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.big_font = pygame.font.SysFont("monospace", 36,
bold=True)
        self.reset()

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
        self.exit_rect = pygame.Rect((COLS-1)*CELL+5, (ROWS-
1)*CELL+5, CELL-10, CELL-10)
        self.start_time = time.time()
        self.elapsed = 0
        self.won = False

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return False
            if event.type == pygame.KEYDOWN and event.key ==
```

```
pygame.K_r:
                self.reset()
        return True

    def update(self):
        if self.won:
            return
        keys = pygame.key.get_pressed()
        self.player.move(keys, self.walls, ROWS, COLS)
        self.elapsed = time.time() - self.start_time
        if self.player.rect.colliderect(self.exit_rect):
            self.won = True

    def draw_maze(self):
        wall_w = 3
        for r in range(ROWS):
            for c in range(COLS):
                x, y = c*CELL, r*CELL
                w = self.walls[r][c]
                if w[0]: pygame.draw.line(self.screen, WALL_COLOR, (x,y),
(x+CELL,y), wall_w)
                if w[1]: pygame.draw.line(self.screen, WALL_COLOR,
(x,y+CELL), (x+CELL,y+CELL), wall_w)
                if w[2]: pygame.draw.line(self.screen, WALL_COLOR,
(x+CELL,y), (x+CELL,y+CELL), wall_w)
                if w[3]: pygame.draw.line(self.screen, WALL_COLOR, (x,y),
(x,y+CELL), wall_w)

    def draw(self):
        self.screen.fill(BG)
        self.draw_maze()
        pygame.draw.rect(self.screen, EXIT_COLOR, self.exit_rect,
border_radius=4)
        ex_label = self.font.render("EXIT", True, (20,80,20))
        self.screen.blit(ex_label, (self.exit_rect.x+2, self.exit_rect.y+4))
        self.player.draw(self.screen)

        hud = pygame.Rect(0, ROWS*CELL, WIDTH, 60)
        pygame.draw.rect(self.screen, (30,30,50), hud)
        time_surf = self.font.render(f"Time: {self.elapsed:.1f}s   R = New
Maze", True, (200,200,200))
        self.screen.blit(time_surf, (10, ROWS*CELL+18))
```

```
        if self.won:
            overlay = pygame.Surface((WIDTH, ROWS*CELL),
pygame.SRCALPHA)
            overlay.fill((0,0,0,120))
            self.screen.blit(overlay, (0,0))
            msg = self.big_font.render(f"Solved in {self.elapsed:.1f}s!", True,
(80,240,80))
            sub = self.font.render("Press R for a new maze", True,
(200,200,200))
            self.screen.blit(msg, (WIDTH//2 - msg.get_width()//2,
ROWS*CELL//2 - 30))
            self.screen.blit(sub, (WIDTH//2 - sub.get_width()//2,
ROWS*CELL//2 + 20))
        pygame.display.flip()

    def run(self):
        running = True
        while running:
            running = self.handle_events()
            self.update()
            self.draw()
            self.clock.tick(FPS)
        pygame.quit()
```

ChatGPT said:

You said:

where to do all this?

ChatGPT said:

You said:

```
import random

CELL = 40  # cell size in pixels

def generate_maze(cols, rows):
    """Recursive backtracker maze generation. Returns 2D grid of
```

```
walls."""
    visited = [[False]*cols for _ in range(rows)]
    # walls: each cell has [N, S, E, W]
    walls = [[[True,True,True,True] for _ in range(cols)] for _ in
range(rows)]

    def neighbors(r, c):
        dirs = [(-1,0,0,1),(1,0,1,0),(0,1,2,3),(0,-1,3,2)]  #
dr,dc,wall_dir,opp_dir
        result = []
        for dr,dc,wd,od in dirs:
            nr,nc = r+dr,c+dc
            if 0<=nr<rows and 0<=nc<cols and not visited[nr][nc]:
                result.append((nr,nc,wd,od))
        return result

    stack = [(0,0)]
    visited[0][0] = True
    while stack:
        r,c = stack[-1]
        nbrs = neighbors(r,c)
        if nbrs:
            nr,nc,wd,od = random.choice(nbrs)
            walls[r][c][wd] = False
            walls[nr][nc][od] = False
            visited[nr][nc] = True
            stack.append((nr,nc))
        else:
            stack.pop()
    return walls

def cell_rect(r, c, import_pygame=None):
    import pygame
    return pygame.Rect(c*CELL, r*CELL, CELL, CELL)
```

ChatGPT said:

You said:

pressing H closes the game window

ChatGPT said:

You said:

```
PS
C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_
Khushi> python main.py
pygame 2.5.2 (SDL 2.28.3, Python 3.11.8)
Hello from the pygame community.
https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File
"C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_
Khushi\main.py", line 5, in <module>
    engine.run()
  File
"C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_
Khushi\game\game_engine.py", line 186, in run
    self.draw()
  File
"C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_
Khushi\game\game_engine.py", line 154, in draw
    self.draw_solution_path()
  File
"C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_
Khushi\game\game_engine.py", line 138, in draw_solution_path
    if (r, c) == (self.player.y // CELL, self.player.x // CELL):
                  ^^^^^^^^^^^^^^^^
AttributeError: 'Player' object has no attribute 'y'
```

ChatGPT said:

You said:

```
import pygame
from game.maze import CELL

SPEED = 3

class Player:
    def __init__(self, r, c):
```

```
        self.r = r
        self.c = c
        x = c*CELL + CELL//2
        y = r*CELL + CELL//2
        self.rect = pygame.Rect(x-10, y-10, 20, 20)
        self.color = (60,120,220)

    def move(self, keys, walls, rows, cols):
        dx, dy = 0, 0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx = -SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx = SPEED
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy = -SPEED
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy = SPEED

        # Wall-aware movement (check cell boundaries)
        new_rect = self.rect.move(dx, 0)
        if not self._hits_wall(new_rect, walls, rows, cols):
            self.rect = new_rect
        new_rect = self.rect.move(0, dy)
        if not self._hits_wall(new_rect, walls, rows, cols):
            self.rect = new_rect

    def _hits_wall(self, rect, walls, rows, cols):
        # Check corners of player rect against wall segments
        for px, py in [(rect.left, rect.top),(rect.right-1,rect.top),
(rect.left,rect.bottom-1),(rect.right-1,rect.bottom-1)]:
            cr = py // CELL
            cc = px // CELL
            if cr < 0 or cr >= rows or cc < 0 or cc >= cols:
                return True
        return False

    def draw(self, screen):
        pygame.draw.ellipse(screen, self.color, self.rect)
```

ChatGPT said:

You said:

works

ChatGPT said:

You said:

```
PS
C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_
Khushi> git status --short
 M game/game_engine.py
PS
C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_
Khushi> git add
Lab4/PS4_PES1UG24AM134_Khushi/game/game_engine.py
warning: could not open directory
'Lab4/PS4_PES1UG24AM134_Khushi/Lab4/PS4_PES1UG24AM134_Khu
shi/game/': No such file or directory
fatal: pathspec
'Lab4/PS4_PES1UG24AM134_Khushi/game/game_engine.py' did not
match any files
PS
C:\Users\Owner\Team4_SE_Mini_Project\Lab4\PS4_PES1UG24AM134_
Khushi>
```

ChatGPT said:

You said:

should i keep two VS window open like one with the lab submission thing and one with the game thing for ease

ChatGPT said:

You said:

```
M  Lab4/PS4_PES1UG24AM134_Khushi/game/game_engine.py
```

ChatGPT said:

You said:

3b90465 (HEAD -> main) Task 1: Add BFS shortest path hint
8c4830c SE Lab 4: Add original Maze Runner game
d9ec174 (origin/main, origin/HEAD) Delete
Lab4/Lab4/PS4_PES1UG24AM129 directory

ChatGPT said:

You said:

found

ChatGPT said:

You said:

```
import pygame
import time
from collections import deque
from game.maze import generate_maze, CELL
from game.player import Player

FPS = 60
BG = (240, 235, 220)
WALL_COLOR = (40, 40, 60)
EXIT_COLOR = (80, 200, 80)
PATH_COLOR = (255, 215, 0)
COLS, ROWS = 15, 13

WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 60


class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Runner")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
```

```
        self.big_font = pygame.font.SysFont("monospace", 36,
bold=True)
        self.reset()

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
        self.exit_rect = pygame.Rect(
            (COLS - 1) * CELL + 5,
            (ROWS - 1) * CELL + 5,
            CELL - 10,
            CELL - 10,
        )
        self.start_time = time.time()
        self.elapsed = 0
        self.won = False
        self.show_path = False
        self.solution_path = self.find_shortest_path()

    def find_shortest_path(self):
        """Find the shortest path from the top-left cell to the exit using
BFS."""
        start = (0, 0)
        target = (ROWS - 1, COLS - 1)
        queue = deque([start])
        parents = {start: None}

        # Direction: row change, column change, current wall, opposite
wall.
        directions = [
            (-1, 0, 0, 1),  # North
            (1, 0, 1, 0),   # South
            (0, 1, 2, 3),   # East
            (0, -1, 3, 2),  # West
        ]

        while queue:
            r, c = queue.popleft()

            if (r, c) == target:
                break
```

```
            for dr, dc, wall_dir, opposite_dir in directions:
                nr, nc = r + dr, c + dc

                if not (0 <= nr < ROWS and 0 <= nc < COLS):
                    continue
                if self.walls[r][c][wall_dir]:
                    continue

                neighbor = (nr, nc)
                if neighbor in parents:
                    continue

                parents[neighbor] = (r, c)
                queue.append(neighbor)

        if target not in parents:
            return []

        path = []
        current = target
        while current is not None:
            path.append(current)
            current = parents[current]

        path.reverse()
        return path

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return False

            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    self.reset()
                elif event.key == pygame.K_h:
                    self.show_path = not self.show_path

        return True

    def update(self):
        if self.won:
```

```
            return

        keys = pygame.key.get_pressed()
        self.player.move(keys, self.walls, ROWS, COLS)
        self.elapsed = time.time() - self.start_time

        if self.player.rect.colliderect(self.exit_rect):
            self.won = True

    def draw_maze(self):
        wall_w = 3
        for r in range(ROWS):
            for c in range(COLS):
                x, y = c * CELL, r * CELL
                w = self.walls[r][c]

                if w[0]:
                    pygame.draw.line(self.screen, WALL_COLOR, (x, y), (x +
CELL, y), wall_w)
                if w[1]:
                    pygame.draw.line(self.screen, WALL_COLOR, (x, y + CELL),
(x + CELL, y + CELL), wall_w)
                if w[2]:
                    pygame.draw.line(self.screen, WALL_COLOR, (x + CELL, y),
(x + CELL, y + CELL), wall_w)
                if w[3]:
                    pygame.draw.line(self.screen, WALL_COLOR, (x, y), (x, y +
CELL), wall_w)

    def draw_solution_path(self):
        """Draw colored squares along the BFS solution path."""
        if not self.show_path:
            return

        for r, c in self.solution_path:
            # Keep the player's current cell and exit visible.
            if (r, c) == (self.player.rect.centery // CELL,
                    self.player.rect.centerx // CELL):
                continue
            if (r, c) == (ROWS - 1, COLS - 1):
                continue
```

```
            rect = pygame.Rect(
                c * CELL + 10,
                r * CELL + 10,
                CELL - 20,
                CELL - 20,
            )
            pygame.draw.rect(self.screen, PATH_COLOR, rect,
border_radius=5)

    def draw(self):
        self.screen.fill(BG)
        self.draw_maze()
        self.draw_solution_path()

        pygame.draw.rect(self.screen, EXIT_COLOR, self.exit_rect,
border_radius=4)
        ex_label = self.font.render("EXIT", True, (20, 80, 20))
        self.screen.blit(ex_label, (self.exit_rect.x + 2, self.exit_rect.y + 4))
        self.player.draw(self.screen)

        hud = pygame.Rect(0, ROWS * CELL, WIDTH, 60)
        pygame.draw.rect(self.screen, (30, 30, 50), hud)
        time_surf = self.font.render(
            f"Time: {self.elapsed:.1f}s   H = Hint   R = New Maze",
            True,
            (200, 200, 200),
        )
        self.screen.blit(time_surf, (10, ROWS * CELL + 18))

        if self.won:
            overlay = pygame.Surface((WIDTH, ROWS * CELL),
pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 120))
            self.screen.blit(overlay, (0, 0))
            msg = self.big_font.render(f"Solved in {self.elapsed:.1f}s!", True,
(80, 240, 80))
            sub = self.font.render("Press R for a new maze", True, (200,
200, 200))
            self.screen.blit(msg, (WIDTH // 2 - msg.get_width() // 2, ROWS
* CELL // 2 - 30))
            self.screen.blit(sub, (WIDTH // 2 - sub.get_width() // 2, ROWS *
CELL // 2 + 20))
```

```
        pygame.display.flip()

    def run(self):
        running = True
        while running:
            running = self.handle_events()
            self.update()
            self.draw()
            self.clock.tick(FPS)

        pygame.quit()
```

this is the current game_engine file make the changes and give complete code

ChatGPT said:

You said:

got a circle that moves with the player and the parts outside the circle are dark

ChatGPT said:

You said:

```
a49f0fa (HEAD -> main) Task 2: Add fog of war
3b90465 Task 1: Add BFS shortest path hint
8c4830c SE Lab 4: Add original Maze Runner game
```

ChatGPT said:

You said:

```
import pygame
import time
from collections import deque
from game.maze import generate_maze, CELL
from game.player import Player
```

```
FPS = 60
BG = (240, 235, 220)
WALL_COLOR = (40, 40, 60)
EXIT_COLOR = (80, 200, 80)
PATH_COLOR = (255, 215, 0)
COLS, ROWS = 15, 13

WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 60


class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Runner")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.big_font = pygame.font.SysFont("monospace", 36,
bold=True)
        self.reset()

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
        self.exit_rect = pygame.Rect(
            (COLS - 1) * CELL + 5,
            (ROWS - 1) * CELL + 5,
            CELL - 10,
            CELL - 10,
        )
        self.start_time = time.time()
        self.elapsed = 0
        self.won = False
        self.show_path = False
        self.solution_path = self.find_shortest_path()

    def find_shortest_path(self):
        """Find the shortest path from start to exit using BFS."""
        start = (0, 0)
        target = (ROWS - 1, COLS - 1)
        queue = deque([start])
```

```
parents = {start: None}

# Row change, column change, current wall, opposite wall.
directions = [
    (-1, 0, 0, 1),  # North
    (1, 0, 1, 0),   # South
    (0, 1, 2, 3),   # East
    (0, -1, 3, 2),  # West
]

while queue:
    r, c = queue.popleft()

    if (r, c) == target:
        break

    for dr, dc, wall_dir, opposite_dir in directions:
        nr, nc = r + dr, c + dc

        if not (0 <= nr < ROWS and 0 <= nc < COLS):
            continue
        if self.walls[r][c][wall_dir]:
            continue

        neighbor = (nr, nc)
        if neighbor in parents:
            continue

        parents[neighbor] = (r, c)
        queue.append(neighbor)

if target not in parents:
    return []

path = []
current = target

while current is not None:
    path.append(current)
    current = parents[current]

path.reverse()
```

```
    return path

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r:
                self.reset()
            elif event.key == pygame.K_h:
                self.show_path = not self.show_path

    return True

def update(self):
    if self.won:
        return

    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)
    self.elapsed = time.time() - self.start_time

    if self.player.rect.colliderect(self.exit_rect):
        self.won = True

def draw_maze(self):
    wall_w = 3

    for r in range(ROWS):
        for c in range(COLS):
            x, y = c * CELL, r * CELL
            w = self.walls[r][c]

            if w[0]:
                pygame.draw.line(
                    self.screen, WALL_COLOR,
                    (x, y), (x + CELL, y), wall_w
                )
            if w[1]:
                pygame.draw.line(
                    self.screen, WALL_COLOR,
```

```
                    (x, y + CELL), (x + CELL, y + CELL), wall_w
                )
            if w[2]:
                pygame.draw.line(
                    self.screen, WALL_COLOR,
                    (x + CELL, y), (x + CELL, y + CELL), wall_w
                )
            if w[3]:
                pygame.draw.line(
                    self.screen, WALL_COLOR,
                    (x, y), (x, y + CELL), wall_w
                )

    def draw_solution_path(self):
        """Draw colored squares along the BFS solution path."""
        if not self.show_path:
            return

        player_cell = (
            self.player.rect.centery // CELL,
            self.player.rect.centerx // CELL,
        )

        for r, c in self.solution_path:
            # Keep the player and exit visible.
            if (r, c) == player_cell:
                continue
            if (r, c) == (ROWS - 1, COLS - 1):
                continue

            rect = pygame.Rect(
                c * CELL + 10,
                r * CELL + 10,
                CELL - 20,
                CELL - 20,
            )
            pygame.draw.rect(
                self.screen, PATH_COLOR, rect, border_radius=5
            )

    def draw_fog_of_war(self):
        """Darken the maze beyond three cells from the player."""
```

```
        fog = pygame.Surface(
            (WIDTH, ROWS * CELL), pygame.SRCALPHA
        )
        fog.fill((0, 0, 0, 235))

        player_center = self.player.rect.center
        reveal_radius = 3 * CELL

        # Transparent circle reveals the area around the player.
        pygame.draw.circle(
            fog,
            (0, 0, 0, 0),
            player_center,
            reveal_radius,
        )

        self.screen.blit(fog, (0, 0))

    def draw(self):
        self.screen.fill(BG)
        self.draw_maze()
        self.draw_solution_path()
        self.draw_fog_of_war()

        # Draw the exit and player above the fog overlay.
        pygame.draw.rect(
            self.screen, EXIT_COLOR, self.exit_rect, border_radius=4
        )
        ex_label = self.font.render("EXIT", True, (20, 80, 20))
        self.screen.blit(
            ex_label,
            (self.exit_rect.x + 2, self.exit_rect.y + 4),
        )
        self.player.draw(self.screen)

        hud = pygame.Rect(0, ROWS * CELL, WIDTH, 60)
        pygame.draw.rect(self.screen, (30, 30, 50), hud)

        time_surf = self.font.render(
            f"Time: {self.elapsed:.1f}s   H = Hint   R = New Maze",
            True,
            (200, 200, 200),
```

```
            )
            self.screen.blit(time_surf, (10, ROWS * CELL + 18))

            if self.won:
                overlay = pygame.Surface(
                    (WIDTH, ROWS * CELL), pygame.SRCALPHA
                )
                overlay.fill((0, 0, 0, 120))
                self.screen.blit(overlay, (0, 0))

                msg = self.big_font.render(
                    f"Solved in {self.elapsed:.1f}s!",
                    True,
                    (80, 240, 80),
                )
                sub = self.font.render(
                    "Press R for a new maze",
                    True,
                    (200, 200, 200),
                )
                self.screen.blit(
                    msg,
                    (WIDTH // 2 - msg.get_width() // 2,
                     ROWS * CELL // 2 - 30),
                )
                self.screen.blit(
                    sub,
                    (WIDTH // 2 - sub.get_width() // 2,
                     ROWS * CELL // 2 + 20),
                )

            pygame.display.flip()

        def run(self):
            running = True

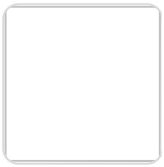
            while running:
                running = self.handle_events()
                self.update()
                self.draw()
                self.clock.tick(FPS)
```

```
pygame.quit()
```

this is the current game_engine make the chnanges and give the complete code

ChatGPT said:

You said:



ChatGPT said:

You said:

exists

ChatGPT said:

You said:

```
 M Lab4/PS4_PES1UG24AM134_Khushi/game/game_engine.py
?? Lab4/PS4_PES1UG24AM134_Khushi/leaderboard.json
```

ChatGPT said:

You said:

```
M  Lab4/PS4_PES1UG24AM134_Khushi/game/game_engine.py
A  Lab4/PS4_PES1UG24AM134_Khushi/leaderboard.json
```

ChatGPT said:

You said:

```
[main 4846617] Task 3: Add timer leaderboard
 2 files changed, 89 insertions(+), 19 deletions(-)
```

```
 create mode 100644
Lab4/PS4_PES1UG24AM134_Khushi/leaderboard.json
```

ChatGPT said:

You said:

```
import pygame
import time
import json
import os
from collections import deque
from game.maze import generate_maze, CELL
from game.player import Player

FPS = 60
BG = (240, 235, 220)
WALL_COLOR = (40, 40, 60)
EXIT_COLOR = (80, 200, 80)
PATH_COLOR = (255, 215, 0)
COLS, ROWS = 15, 13

WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 60
LEADERBOARD_FILE = os.path.join(
    os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
    "leaderboard.json",
)


class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Runner")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.big_font = pygame.font.SysFont("monospace", 36,
bold=True)
        self.small_font = pygame.font.SysFont("monospace", 17)
        self.reset()
```

```
    def load_leaderboard(self):
        """Load saved completion times from JSON."""
        try:
            with open(LEADERBOARD_FILE, "r", encoding="utf-8") as file:
                data = json.load(file)
            if isinstance(data, list):
                valid_times = [
                    float(entry)
                    for entry in data
                    if isinstance(entry, (int, float))
                    and not isinstance(entry, bool)
                    and entry >= 0
                ]
                return sorted(valid_times)[:5]
        except (FileNotFoundError, json.JSONDecodeError, OSError,
ValueError):
            pass
        return []

    def save_completion_time(self):
        """Save the latest completion time and retain the five fastest
times."""
        leaderboard = self.load_leaderboard()
        leaderboard.append(round(self.elapsed, 2))
        leaderboard = sorted(leaderboard)[:5]

        try:
            with open(LEADERBOARD_FILE, "w", encoding="utf-8") as file:
                json.dump(leaderboard, file, indent=4)
        except OSError as error:
            print(f"Could not save leaderboard: {error}")

        self.leaderboard = leaderboard

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
        self.exit_rect = pygame.Rect(
            (COLS - 1) * CELL + 5,
            (ROWS - 1) * CELL + 5,
            CELL - 10,
            CELL - 10,
```

```
        )
        self.start_time = time.time()
        self.elapsed = 0
        self.won = False
        self.completion_recorded = False
        self.show_path = False
        self.solution_path = self.find_shortest_path()
        self.leaderboard = self.load_leaderboard()

    def find_shortest_path(self):
        """Find the shortest path from start to exit using BFS."""
        start = (0, 0)
        target = (ROWS - 1, COLS - 1)
        queue = deque([start])
        parents = {start: None}

        directions = [
            (-1, 0, 0, 1),  # North
            (1, 0, 1, 0),   # South
            (0, 1, 2, 3),   # East
            (0, -1, 3, 2),  # West
        ]

        while queue:
            r, c = queue.popleft()

            if (r, c) == target:
                break

            for dr, dc, wall_dir, opposite_dir in directions:
                nr, nc = r + dr, c + dc

                if not (0 <= nr < ROWS and 0 <= nc < COLS):
                    continue
                if self.walls[r][c][wall_dir]:
                    continue

                neighbor = (nr, nc)
                if neighbor in parents:
                    continue

                parents[neighbor] = (r, c)
```

```
                queue.append(neighbor)

        if target not in parents:
            return []

        path = []
        current = target
        while current is not None:
            path.append(current)
            current = parents[current]

        path.reverse()
        return path

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return False

            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    self.reset()
                elif event.key == pygame.K_h:
                    self.show_path = not self.show_path

        return True

    def update(self):
        if self.won:
            return

        keys = pygame.key.get_pressed()
        self.player.move(keys, self.walls, ROWS, COLS)
        self.elapsed = time.time() - self.start_time

        if self.player.rect.colliderect(self.exit_rect):
            self.elapsed = time.time() - self.start_time
            self.won = True

            if not self.completion_recorded:
                self.save_completion_time()
                self.completion_recorded = True
```

```
def draw_maze(self):
    wall_w = 3

    for r in range(ROWS):
        for c in range(COLS):
            x, y = c * CELL, r * CELL
            w = self.walls[r][c]

            if w[0]:
                pygame.draw.line(
                    self.screen, WALL_COLOR,
                    (x, y), (x + CELL, y), wall_w
                )
            if w[1]:
                pygame.draw.line(
                    self.screen, WALL_COLOR,
                    (x, y + CELL), (x + CELL, y + CELL), wall_w
                )
            if w[2]:
                pygame.draw.line(
                    self.screen, WALL_COLOR,
                    (x + CELL, y), (x + CELL, y + CELL), wall_w
                )
            if w[3]:
                pygame.draw.line(
                    self.screen, WALL_COLOR,
                    (x, y), (x, y + CELL), wall_w
                )

def draw_solution_path(self):
    """Draw colored squares along the BFS solution path."""
    if not self.show_path:
        return

    player_cell = (
        self.player.rect.centery // CELL,
        self.player.rect.centerx // CELL,
    )

    for r, c in self.solution_path:
        if (r, c) == player_cell:
```

```
            continue
        if (r, c) == (ROWS - 1, COLS - 1):
            continue

        rect = pygame.Rect(
            c * CELL + 10,
            r * CELL + 10,
            CELL - 20,
            CELL - 20,
        )
        pygame.draw.rect(
            self.screen, PATH_COLOR, rect, border_radius=5
        )

    def draw_fog_of_war(self):
        """Darken the maze beyond three cells from the player."""
        fog = pygame.Surface(
            (WIDTH, ROWS * CELL), pygame.SRCALPHA
        )
        fog.fill((0, 0, 0, 235))

        pygame.draw.circle(
            fog,
            (0, 0, 0, 0),
            self.player.rect.center,
            3 * CELL,
        )

        self.screen.blit(fog, (0, 0))

    def draw(self):
        self.screen.fill(BG)
        self.draw_maze()
        self.draw_solution_path()
        self.draw_fog_of_war()

        pygame.draw.rect(
            self.screen, EXIT_COLOR, self.exit_rect, border_radius=4
        )
        ex_label = self.font.render("EXIT", True, (20, 80, 20))
        self.screen.blit(
            ex_label,
```

```
            (self.exit_rect.x + 2, self.exit_rect.y + 4),
        )
        self.player.draw(self.screen)

        hud = pygame.Rect(0, ROWS * CELL, WIDTH, 60)
        pygame.draw.rect(self.screen, (30, 30, 50), hud)

        time_surf = self.font.render(
            f"Time: {self.elapsed:.1f}s   H = Hint   R = New Maze",
            True,
            (200, 200, 200),
        )
        self.screen.blit(time_surf, (10, ROWS * CELL + 18))

        if self.won:
            overlay = pygame.Surface(
                (WIDTH, ROWS * CELL), pygame.SRCALPHA
            )
            overlay.fill((0, 0, 0, 190))
            self.screen.blit(overlay, (0, 0))

            msg = self.big_font.render(
                f"Solved in {self.elapsed:.1f}s!",
                True,
                (80, 240, 80),
            )
            self.screen.blit(
                msg,
                (WIDTH // 2 - msg.get_width() // 2, 12),
            )

            title = self.font.render(
                "TOP 5 TIMES", True, (255, 215, 0)
            )
            self.screen.blit(
                title,
                (WIDTH // 2 - title.get_width() // 2, 58),
            )

            if self.leaderboard:
                for index, completion_time in enumerate(self.leaderboard):
                    line = self.small_font.render(
```

```
                    f"{index + 1}. {completion_time:.2f} seconds",
                    True,
                    (255, 255, 255),
                )
                self.screen.blit(
                    line,
                    (WIDTH // 2 - line.get_width() // 2, 88 + index * 25),
                )
        else:
            line = self.small_font.render(
                "No completion times yet",
                True,
                (255, 255, 255),
            )
            self.screen.blit(
                line,
                (WIDTH // 2 - line.get_width() // 2, 95),
            )

        sub = self.small_font.render(
            "Press R for a new maze",
            True,
            (200, 200, 200),
        )
        self.screen.blit(
            sub,
            (WIDTH // 2 - sub.get_width() // 2, ROWS * CELL - 35),
        )

    pygame.display.flip()

def run(self):
    running = True

    while running:
        running = self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)

    pygame.quit()
```

ChatGPT said:

You said:

done

ChatGPT said:

You said:

everything is working

ChatGPT said:

You said:

M Lab4/PS4_PES1UG24AM134_Khushi/game/game_engine.py
M Lab4/PS4_PES1UG24AM134_Khushi/leaderboard.json

ChatGPT said:

You said:

[main e38502d] Task 4: Add difficulty selection
1 file changed, 169 insertions(+), 58 deletions(-)

ChatGPT said:

You said:

M Lab4/PS4_PES1UG24AM134_Khushi/leaderboard.json
S C:\Users\Owner\Team4_SE_Mini_Project> git log -5 --oneline
e38502d (HEAD -> main) Task 4: Add difficulty selection
4846617 Task 3: Add timer leaderboard
a49f0fa Task 2: Add fog of war
3b90465 Task 1: Add BFS shortest path hint
8c4830c SE Lab 4: Add original Maze Runner game

ChatGPT said:

You said:

```
[
    12.15,
    74.26
]
PS
```

ChatGPT said:

You said:

```
[main 627ccf5] Update maze completion leaderboard
 1 file changed, 1 insertion(+)
```

ChatGPT said:

You said:

```
On branch main
Your branch is ahead of 'origin/main' by 6 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

ChatGPT said:

You said:

```
Enumerating objects: 45, done.
Counting objects: 100% (45/45), done.
Delta compression using up to 8 threads
Compressing objects: 100% (38/38), done.
Writing objects: 100% (43/43), 10.17 KiB | 242.00 KiB/s, done.
Total 43 (delta 19), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (19/19), completed with 1 local object.
```

To https://github.com/Khushi-patel05/Team4_SE_Mini_Project.git
d9ec174..627ccf5 main -> main

ChatGPT said:

You said:

i am getting some error i am not able to take video of the game after changes

ChatGPT said:

You said:

videos recorded

ChatGPT said:

You said:

videos commited now next

ChatGPT said:

You said:

let's export the AI chat history

ChatGPT said: